Speaking of debugging and possible failures, this interaction really should have a test.

System Testing the Checkout Flow

In *Iteration G3: Testing Our JavaScript Functionality*, you wrote a system test that uses a real browser to simulate user interaction. To test the entire flow of checking out, communicating with the payment processor, and sending an email, you'll add a second test.

To test the full, end-to-end workflow, including execution of Active Jobs, you want to do the following:

1. Add a book to the cart.
2. Fill in the checkout form completely (including selecting a pay type).
3. Submit the order.
4. Process all background jobs.
5. Check that the order was created properly.
6. Check that email was sent.

You should already be familiar with how to write most parts of this test. Processing background jobs and checking mail, however, are new. Rails provides helpers for us, so the test will be short and readable when you're done. One of those helpers is available by mixing in the `ActiveJob::TestHelper` module:

rails80/depot_qb/test/system/orders_test.rb

```
  class OrdersTest < ApplicationSystemTestCase
»   include ActiveJob::TestHelper
```

This provides the method `perform_enqueued_jobs`, which you'll use in your test:

rails80/depot_qb/test/system/orders_test.rb

```
  test "check order and delivery" do
    LineItem.delete_all
```